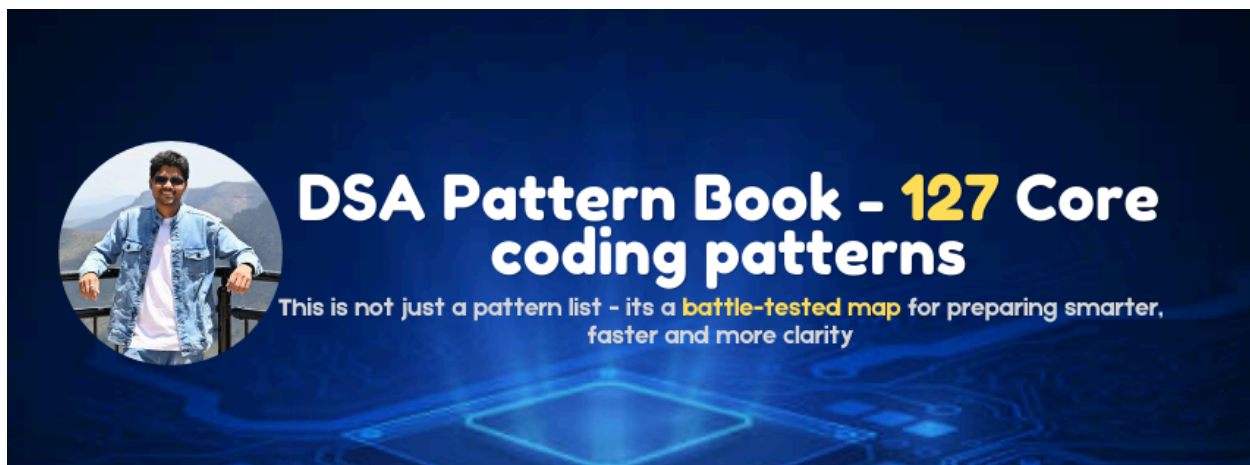




127 Core Patterns by Lavakumar Thatisetti - for 1:1

⚙️ DSA Pattern Book – Core Format

Welcome to the **127 Core DSA Patterns** – your go-to handbook for cracking coding interviews with clarity, confidence, and speed.

This book breaks down the most **frequently asked, high-impact**, and **FAANG-loved** Data Structures & Algorithms patterns into a repeatable, easy-to-digest format.

For each pattern, you'll get:

- 🧠 **Pattern Name** – What it's called and where it's used
- 📌 **Core Logic** – The heart of how the pattern works
- 🔄 **Variations** – Common twists that show up in real interviews
- ✨ **Bonus** – Company tags showing where it's most loved (e.g., *Hot at Google, Meta favorite*)
- 🔥 **Example Problems** – Leetcode, GFG, and Codeforces references to master it in practice

Whether you're preparing for your first coding round, brushing up for a FAANG interview — this book is built to guide you, step-by-step.

Author: https://bondin.io/lavakumar_thatisetti



Table of Contents

Section	Title
1	Array & String Patterns – Pointers, Windows, Prefixes, and Subarrays
2	Linked Lists – Classic recursion, pointer tricks, and state-saving
3	Heaps, Stacks & Queues – Real-time ranking, expression evaluation, span problems. HashMap & TreeMap Toolkits – Frequency, bucketing, ordering, and fast lookups
4	Recursion and Divide and Conquer Patterns
5a	Dynamic Programming (Part 1) – Subset, Count, Cost, Sequence, 0/1 Knapsack
5b	Dynamic Programming (Part 2) – Palindromes, Digit DP, Bitmask, Stock, Regex
6	Tree Traversals
7	Graphs & Backtracking – DFS, BFS, Union-Find, Topo Sort, and Constraint Solvers
8	Greedy Patterns
9	Advanced Strings & Tries – Rolling hash, KMP, encoding, builder patterns, Autocomplete, word search, bitwise tries. Segment Trees – Range queries, lazy propagation, real-time updates
10	Matrix, Geometry and Simulation Patterns
11	Wildcard + Bonus Patterns – Rare but powerful topics like Meet-in-the-Middle, Sparse Table
12	OOP & Design-Based Patterns – System-level patterns, modular coding

SECTION 1: ARRAY & STRING PATTERNS

Let's begin with 15 rock-solid foundational patterns from Arrays and Strings.

1. Two Pointers (Opposite Direction)

 Use one pointer from the start and another from the end, converging toward the center.

 **Why it matters:**

- Solves many **$O(n)$** problems that naïvely take $O(n^2)$ using brute force.
- Great for **sorted arrays, symmetric comparisons, and pair matching**.
- Used in **Amazon** and **Facebook** for string/array logic.

 Variants:

- Palindromes
- Sum of two numbers
- Array partitioning

 Examples:

- [Leetcode 167 - Two Sum II \(Input array is sorted\)](#)
- [Leetcode 125 - Valid Palindrome](#)
- [GFG - Array Pair Sum Divisibility](#)

2. Two Pointers (Same Direction / Fast-Slow)

 One pointer moves faster than the other to compare relative positions.

 **Why it matters:**

- Crucial in **cycle detection, finding midpoints, removing duplicates**.
- Appears often in **LinkedIn, Meta, and service logic-based interviews**.

 Variants:

- Find middle of a list
- Remove duplicates in-place

- Detect cycles (linked lists, numbers)

🔥 Examples:

- [Leetcode 26 - Remove Duplicates from Sorted Array](#)
 - [Leetcode 141 - Linked List Cycle](#)
 - [Leetcode 876 - Middle of Linked List](#)
-

3. 🧠 Sliding Window (Fixed Size)

📌 Move a window of fixed length across the array to calculate something.

🎯 **Why it matters:**

- Appears in **real-time analytics**, streaming, and **rate-limiting** systems.
- Loved in **Microsoft**, **Amazon**, and optimization-focused rounds.

🔄 Variants:

- Max sum of k-sized subarray
- Min/avg values
- Windowed product

🔥 Examples:

- [Leetcode 643 - Maximum Average Subarray I](#)
 - [GFG - Max Sum Subarray of Size K](#)
 - [Leetcode 1456 - Max Number of Vowels in a Substring](#)
-

4. 🧠 Sliding Window (Dynamic Size)

📌 Expand the window until a condition breaks, then shrink from left.

🎯 **Why it matters:**

- Appears in **longest substring**, bandwidth throttling, compression
- Common in **Google + Uber** interviews

🔄 Variants:

- Longest substring with unique characters
- Min window containing pattern
- Count substrings with at most K distinct characters

🔥 Examples:

- [Leetcode 3 - Longest Substring Without Repeating Characters](#)
- [Leetcode 76 - Minimum Window Substring](#)
- [Leetcode 159 - Longest Substring with At Most Two Distinct Characters](#)

5. 🧠 Prefix Sum / Difference Array

📌 Store running totals or range diffs to **answer subarray questions in O(1)**.

🎯 Why it matters:

- Efficient in queries like: "What's the sum from index i to j?"
- Used in **Meta + Airbnb** to optimize brute force solutions.

🔄 Variants:

- Subarray sums
- Range sum queries
- Difference arrays for range updates

🔥 Examples:

- [Leetcode 560 - Subarray Sum Equals K](#)
- [Leetcode 724 - Find Pivot Index](#)
- [Leetcode 303 - Range Sum Query](#)

6. 🧠 Hashing for Lookup

📌 Use **HashMap** or **HashSet** for fast O(1) checks.

🎯 Why it matters:

- Great for **duplicate detection**, **frequency counts**, and **complement lookup**

- Often used in **Google** and **Amazon** first-rounds

 Variants:

- 2Sum
- Check for duplicates
- Frequency match/anagram check

 Examples:

- [Leetcode 1 - Two Sum](#)
 - [Leetcode 217 - Contains Duplicate](#)
 - [Leetcode 383 - Ransom Note](#)
-

7. Binary Search (Standard)

 Halve the search space in a sorted array to find a target efficiently.

 **Why it matters:**

- Foundational for system design, file systems, and **FAANG API efficiency**
- Evaluates **index management and recursion handling**.

 Variants:

- First/last position
- Target in rotated array
- Recursive vs iterative

 Examples:

- [Leetcode 704 - Binary Search](#)
 - [Leetcode 34 - Find First and Last Position](#)
 - [Leetcode 35 - Search Insert Position](#)
-

8. Binary Search (Answer Space)

 Instead of searching in data, search over **possible values/answers**.

Why it matters:

- Common in **optimization and allocation problems**
- Loved by **Google, Uber, and Amazon** for thinking beyond simple binary search.

Variants:

- Koko Eating Bananas
- Min largest sum in K parts
- Smallest divisor under threshold

Examples:

- [Leetcode 410 - Split Array Largest Sum](#)
 - [Leetcode 875 - Koko Eating Bananas](#)
 - [Leetcode 1283 - Smallest Divisor Given a Threshold](#)
-

9. Two Sum Family

 Find combinations that sum to a target (2Sum, 3Sum, 4Sum).

Why it matters:

- Tests **array sorting, pointer logic, and duplicate handling**
- Interviewers use it for step-up questions in **Facebook, Adobe, Microsoft**

Variants:

- Find any valid pair(s)
- All unique combinations
- KSum generalizations

Examples:

- [Leetcode 1 - Two Sum](#)
 - [Leetcode 15 - 3Sum](#)
 - [Leetcode 18 - 4Sum](#)
-

10. 🧠 Monotonic Stack

📌 Stack that maintains **increasing or decreasing order** to find next/prev elements

🎯 **Why it matters:**

- Efficient $O(n)$ solutions to **span, histogram, temperature**, and **next greater** problems
- Interviewers check **invariant maintenance + optimization skills**

🔄 Variants:

- Next greater/smaller
- Span problems
- Histogram max area

🔥 Examples:

- [Leetcode 739 - Daily Temperatures](#)
 - [Leetcode 84 - Largest Rectangle in Histogram](#)
 - [Leetcode 496 - Next Greater Element I](#)
-

11. 🧠 Merge Intervals

📌 Sort intervals and merge overlapping ones

🎯 **Why it matters:**

- Used in **calendar merging, scheduling, memory allocation**
- Evaluates sorting, merging, and iteration control

🔄 Variants:

- Insert/merge interval
- Count overlaps
- Remove overlaps

🔥 Examples:

- [Leetcode 56 - Merge Intervals](#)

- [Leetcode 57 - Insert Interval](#)
 - [Leetcode 435 - Non-overlapping Intervals](#)
-

12. 🧠 In-place Array Rearrangement

📌 Reorder or mark arrays using indices — without extra space.

🎯 **Why it matters:**

- Appears in **cyclic sort**, duplicate finding, missing number
- Forces candidates to **optimize space and reuse input array**

🔄 Variants:

- Swap-based positioning
- Cyclic sort
- Mark values as visited

🔥 Examples:

- [Leetcode 448 - Find All Disappeared Numbers](#)
 - [Leetcode 287 - Find Duplicate Number](#)
 - [Leetcode 41 - First Missing Positive](#)
-

13. 🧠 Kadane's Algorithm (Max Subarray Sum)

📌 Track current max, reset when negative to find best subarray

🎯 **Why it matters:**

- Classic DP-based array problem
- Asked repeatedly in **Google, Apple, Amazon**

🔄 Variants:

- Max subarray (1D, circular)
- Include/exclude one element
- Max average

🔥 Examples:

- [Leetcode 53 - Maximum Subarray](#)
 - [Leetcode 918 - Maximum Sum Circular Subarray](#)
 - [Leetcode 1191 - K-Concatenation Maximum Sum](#)
-

14. 🧠 String Reversal Techniques

📌 Reverse full or partial strings using pointer techniques

🎯 Why it matters:

- Appears in **encoding, decoding, reversing word order, rotation**
- Interviewers check **in-place manipulation and loop control**

🔄 Variants:

- Reverse full string
- Reverse words in sentence
- Rotate string

🔥 Examples:

- [Leetcode 344 - Reverse String](#)
 - [Leetcode 151 - Reverse Words in a String](#)
 - [Leetcode 186 - Reverse Words in a String II](#)
-

15. 🧠 Anagram Grouping

📌 Normalize words (sort or frequency) to group anagrams

🎯 Why it matters:

- Appears in **search engines, spell checkers, natural language tools**
- Great for testing **string maps and grouping logic**

🔄 Variants:

- Group all anagrams

- Check if two words are anagrams
- Count total anagram groups

🔥 Examples:

- [Leetcode 49 - Group Anagrams](#)
- [Leetcode 242 - Valid Anagram](#)
- [GFG - Print All Anagram Groups Together](#)

SECTION 2: LINKED LIST PATTERNS

16. Reverse Linked List

 Reverse the links in a singly linked list (iterative or recursive).

 **Why it matters:**

- Appears in almost every **LinkedIn, Meta, Amazon** round as a base concept.
- Core to many **composite problems** (e.g., palindrome checks, group reversals).

 Variants:

- Full reverse
- Reverse sublist (between m and n)
- Reverse every k nodes

 Examples:

- [Leetcode 206 - Reverse Linked List](#)
- [Leetcode 92 - Reverse Linked List II](#)
- [Leetcode 25 - Reverse Nodes in k-Group](#)

17. Detect Cycle in Linked List

 Use **Floyd's Tortoise & Hare** to detect cycles efficiently.

 **Why it matters:**

- Classic question used to test **pointer manipulation and fast-slow technique**.
- Important in memory leak detection and **looped system chains**.

 Variants:

- Detect if a cycle exists
- Find the node where cycle starts
- Calculate cycle length

 Examples:

- [Leetcode 141 - Linked List Cycle](#)
 - [Leetcode 142 - Linked List Cycle II](#)
-

18. 🧠 Merge Two Linked Lists

📌 Combine two sorted linked lists into one sorted list.

🎯 **Why it matters:**

- Real-world app in **calendar merging, scheduling**, stream processing.
- Often seen in **Microsoft and Uber** rounds.

🔄 Variants:

- Merge two sorted lists
- Merge k sorted lists
- Merge in-place vs allocate new nodes

🔥 Examples:

- [Leetcode 21 - Merge Two Sorted Lists](#)
 - [Leetcode 23 - Merge K Sorted Lists](#)
-

19. 🧠 Find Middle of Linked List

📌 Use fast-slow pointers to get middle node.

🎯 **Why it matters:**

- Used as a **pre-processing step** in many divide & conquer algorithms on linked lists.
- Popular in **palindrome, sorting**, and **merge** logic.

🔄 Variants:

- Middle node
- Pre-mid (for odd-sized lists)
- Use in fast/slow operations

🔥 Example:

- [Leetcode 876 - Middle of Linked List](#)
-

20. 🧠 Remove N-th Node From End

📌 Use two pointers to locate and delete the node.

🎯 **Why it matters:**

- Combines **edge case handling** + **pointer distance trick**.
- Tricky off-by-one challenges — loved in **Google & Amazon** rounds.

🔄 Variants:

- Remove nth node
- One-pass vs two-pass solutions
- Use dummy head to handle edge cases

🔥 Example:

- [Leetcode 19 - Remove Nth Node From End](#)
-

21. 🧠 Palindrome Linked List

📌 Reverse the second half, compare with first.

🎯 **Why it matters:**

- Excellent combo of **slow/fast pointers, reverse, compare**
- Appears in **Meta + Amazon** for combining multiple techniques.

🔄 Variants:

- Check palindrome
- Restore original list
- Use stack if not allowed to modify

🔥 Example:

- [Leetcode 234 - Palindrome Linked List](#)

◆ SECTION 3: HEAPS, HASHMAPS, STACKS, and STRUCTURE-BASED PATTERNS

🔧 HEAP / PRIORITY QUEUE PATTERNS

22. 🧠 Top K Elements using Min/Max Heap

📌 Maintain a **min-heap of size K** to track top K largest or most frequent elements.

🎯 Core Idea: Insert elements, remove smallest when over size

🌟 **Why it matters:**

- Frequently used in **streaming, log analysis, and dashboards**. Very common in **Amazon and Netflix**.
- **Loved by:** Amazon, Google, Netflix

🔄 Variants:

- Top K frequent
- Kth largest/smallest
- Sort streams

🔥 Examples:

- [Leetcode 215 - Kth Largest Element in an Array](#)
 - [Leetcode 347 - Top K Frequent Elements](#)
 - [Leetcode 703 - Kth Largest in a Stream](#)
-

23. 🧠 Merge K Sorted Lists (Min Heap)

📌 Merge multiple sorted arrays/lists using a min-heap to pick the smallest across heads.

🌟 **Why it matters:**

- Core pattern for **log merging, stream processing, external sorting**.
- 🔍 **Loved by:** Google, Bloomberg, Meta

🔄 Variants:

- External sorting
- Calendar merging
- Log merging

🔥 Examples:

- [Leetcode 23 - Merge K Sorted Lists](#)
 - [Leetcode 373 - Find K Pairs with Smallest Sums](#)
 - [Leetcode 632 - Smallest Range Covering Elements from K Lists](#)
-

24. 🧠 Sliding Window Median / Max / Min

📌 Use 2 Heaps (or TreeMap) to dynamically manage a sliding window

🎯 Core: Keep halves balanced or use custom sorting

💡 Why: Tricky combo of heap + window logic

🌟 **Why it matters:**

- Powerful in **time-series, real-time analytics, and pricing systems.**
- 🔍 **Hot in:** Bloomberg, Google, Stripe

🔄 Variants:

- Median in window
- Min/max of K elements

🔥 Examples:

- [Leetcode 295 - Find Median from Data Stream](#)
- [Leetcode 480 - Sliding Window Median](#)
- [Leetcode 239 - Sliding Window Maximum](#)

STACK & QUEUE PATTERNS

25. Next Greater Element (Monotonic Stack)

 Use a decreasing stack to efficiently track **next/previous greater or smaller** items.

 **Why it matters:**

- Helps reduce brute force $O(n^2)$ to $O(n)$. Common in **UI layout, histogram analysis**.
-  Core: Maintain **invariant order** while scanning
-  **Loved by:** Meta, Adobe, Amazon

 Variants:

- Stock span
- Histogram area
- Circular arrays

 Examples:

- [Leetcode 496 - Next Greater Element I](#)
- [Leetcode 503 - Next Greater Element II](#)
- [Leetcode 901 - Online Stock Span](#)

26. Valid Parentheses / Expression Matching

 Use stack to match open/close characters or build properly nested structures

 Core: Push/pop stack for structural checks

 **Why it matters:**

Powers **parsers, calculators, bracket matchers**, commonly asked in **entry-mid rounds**.

- Classic parsing trick
- Powers **parsers, calculators, bracket matchers**, commonly asked in **entry-mid rounds**

- 🔍 **Hot at:** Amazon, Microsoft, Apple

🔄 Variants:

- Min removal
- Balanced indexes
- Expression evaluation

🔥 Examples:

- Leetcode 20 - Valid Parentheses
- Leetcode 1249 - Minimum Remove to Make Valid
- Leetcode 1541 - Minimum Insertions to Balance a String

27. 🧠 Implement Stack/Queue using Other

📌 Build one structure using another (stack using queue or vice versa)

🎯 Core: Layering ops like pop/push with reverse logic

🌟 **Why it matters:**

- Frequently asked in entry-level rounds
- Interviewers want to test **understanding of data structure internals**
- 🔍 **Loved by:** Amazon, Apple (early rounds)

🔄 Variants:

- Two-stack queue
- Min-stack
- Queue with constant time max

🔥 Examples:

- Leetcode 232 - Implement Queue using Stacks
- Leetcode 225 - Implement Stack using Queues
- Leetcode 155 - Min Stack



HASHMAP / TREEMAP PATTERNS

28. 🧠 Frequency Counters (Char/Word/Num)

📌 Track frequency of elements using Hashmaps.

🎯 Core: Easy to maintain state across iteration

★ Why it matters:

- Used in grouping, matching, sliding window
- Base for **anagram detection, majority element, char-count DP**.
- 🔍 **Hot in:** Amazon, Adobe, Walmart

🔄 Variants:

- Anagrams
- Majority
- Duplicate detection

🔥 Examples:

- Leetcode 387 - First Unique Character
- Leetcode 451 - Sort Characters by Frequency
- Leetcode 169 - Majority Element

29. 🧠 Grouping and Bucketing using HashMap

📌 Group elements by a key like frequency or transformed string

🎯 Core: Grouping → sorting/selection → map-based finalization

★ Why it matters:

- Used in grouping similar items
- Core for **clustering, histogram bins, grouping problems**
- 🔍 **Loved by:** Facebook, Google, Uber

🔄 Variants:

- Group anagrams
- Group by value
- Word frequency

🔥 Examples:

- [Leetcode 49 - Group Anagrams](#)
- [Leetcode 347 - Top K Frequent Words](#)
- [Leetcode 692 - Top K Frequent Words](#)

30. 🧠 Ordered HashMap (TreeMap, LinkedHashMap)

📌 Keep insertion order or key-order when processing

🎯 Core: Maintain sorted keys or recency-based access

🌟 **Why it matters:** Enables **range queries, eviction systems, LRU logic.**

🔍 **Loved by:** Dropbox, Bloomberg, Google

🔄 Variants:

- LRU Cache
- Min window substring
- Median of sliding window

🔥 Examples:

- [Leetcode 146 - LRU Cache](#)
- [Leetcode 480 - Sliding Window Median](#)
- [Leetcode 76 - Minimum Window Substring](#)

SECTION 4: RECURSION + DIVIDE & CONQUER PATTERNS

31. Classic Recursion (Top-Down Tree of Decisions)

 Solve problems by recursively branching into all choices

 Core Idea: Build a **recursive tree of states**, handle base/recursive cases

 Why: Builds intuition for DP, backtracking, and more

 Variants:

- Sum of numbers
- String reverse
- Factorials, Fibonacci

 Examples:

- [Leetcode 70 - Climbing Stairs](#)
 - [Leetcode 104 - Max Depth of Binary Tree](#)
 - [Leetcode 344 - Reverse String \(Recursive\)](#)
 - [Leetcode 509 - Fibonacci Number](#)
-

32. Recursion with Memoization (Top-Down DP)

 Store results of subproblems in a cache to avoid recomputation

 Core Idea: Add a **cache map** (or array) to standard recursion

 Why: Makes naive recursive brute force efficient

 Variants:

- Fibonacci
- Subset sum
- Grid paths

 Examples:

- [Leetcode 62 - Unique Paths](#)

- [Leetcode 1137 - N-th Tribonacci Number](#)
 - [Leetcode 198 - House Robber](#)
 - [Leetcode 91 - Decode Ways](#)
-

33. 🧠 Divide & Conquer (Binary Split Recursion)

📌 Break the problem in **two halves**, solve, and merge

🎯 Core Idea: Recursively divide input (often $\text{mid} = (l + r)/2$)

💡 Why: Enables $\log(n)$ depth recursion trees and parallelization

🔄 Variants:

- Sorting
- Searching
- Counting inversions

🔥 Examples:

- [Leetcode 23 - Merge K Sorted Lists](#)
 - [Leetcode 53 - Maximum Subarray \(Kadane + Divide\)](#)
 - [Leetcode 241 - Different Ways to Add Parentheses](#)
-

34. 🧠 Merge Sort

📌 A sorting algorithm that recursively splits and merges arrays

🎯 Core Idea: Split array in halves → recursively sort → merge

💡 Why: Stable $O(n \log n)$ sort that works on linked lists too

🔄 Variants:

- Count reverse pairs
- Sort linked list
- Merge intervals

🔥 Examples:

- [Leetcode 148 - Sort List](#)

- [Leetcode 493 - Reverse Pairs](#)
 - [GFG - Merge Sort for Linked List](#)
-

35. 🧠 Quick Sort (Partitioning Recursion)

📌 Choose a pivot, partition, and recursively sort both sides

🎯 Core: Everything left of pivot is smaller; right is larger

💡 Why: Commonly used in real-world systems

🔄 Variants:

- Randomized quicksort
- In-place sorting
- Kth smallest (partial quicksort)

🔥 Examples:

- [Leetcode 912 - Sort an Array](#)
 - [GFG - Quick Sort Implementation](#)
-

36. 🧠 Binary Search (Divide & Search)

📌 Halve the search space each step

🎯 Core: Check mid → reduce search interval

💡 Why: One of the most used divide & conquer techniques

🔄 Variants:

- First/last position
- Rotated arrays
- Minimum in range

🔥 Examples:

- [Leetcode 33 - Search in Rotated Sorted Array](#)
- [Leetcode 34 - First and Last Position](#)
- [Leetcode 74 - Search a 2D Matrix](#)

37. 🧠 Modified Binary Search (Search on Answer)

📌 Binary search where you search on the **answer space**, not the array

🎯 Core: Search between range of possible answers

💡 Why: Ideal for "min/max possible value" type questions

🔄 Variants:

- Min max of subarray
- Koko Eating Bananas
- Aggressive cows (GFG classic)

🔥 Examples:

- [Leetcode 875 - Koko Eating Bananas](#)
 - [Leetcode 410 - Split Array Largest Sum](#)
 - [Leetcode 1283 - Smallest Divisor Given Threshold](#)
-

38. 🧠 Recursive Backtracking

📌 Try every path recursively; backtrack when invalid

🎯 Core: Place choice → recurse → undo choice (backtrack)

💡 Why: Solves exponential choice trees cleanly

🔄 Variants:

- Permutations
- Sudoku/N-Queens
- Expression parsing

🔥 Examples:

- [Leetcode 46 - Permutations](#)
- [Leetcode 77 - Combinations](#)
- [Leetcode 37 - Sudoku Solver](#)
- [Leetcode 282 - Expression Add Operators](#)

39. 🧠 Recursive DFS in Graphs and Grids

📌 Traverse graphs/grid using recursion, marking visited

🎯 Core: Explore deeply → backtrack

💡 Why: Appears in grid island problems, component checks

🔄 Variants:

- Cycle detection
- Flood fill
- Maze solving

🔥 Examples:

- [Leetcode 200 - Number of Islands](#)
 - [Leetcode 695 - Max Area of Island](#)
 - [Leetcode 733 - Flood Fill](#)
-

40. 🧠 Tree Recursion (Preorder, Postorder Computation)

📌 Compute values in trees recursively (height, sum, LCA, etc.)

🎯 Core: Base case = null, recursive = left/right

💡 Why: Most tree logic is recursive

🔄 Variants:

- Max depth
- Tree diameter
- Path sums

🔥 Examples:

- [Leetcode 543 - Diameter of Binary Tree](#)
- [Leetcode 124 - Binary Tree Maximum Path Sum](#)
- [Leetcode 112 - Path Sum](#)

🧠 SECTION 5: DYNAMIC PROGRAMMING PATTERNS (Part 1 – Core 0/1 Subsets & Decisions)

These are the **heart of DP in interviews**. Let's go deep but crisp.

41. 🧠 DP – 0/1 Knapsack Pattern

📌 You're given items with weights & values and must **choose or skip** each — without exceeding capacity.

🎯 Core idea: "For each item → take it or leave it."

✨ Reusable for problems involving **maximum sum, limited capacity, and inclusion/exclusion decisions**

🎯 **Why it matters:**

- The foundation of almost every **decision-based DP**.
- Teaches core principles like **overlapping subproblems and state choices**.

💡 **Loved by Amazon & Flipkart** for operations-focused roles.

🔥 Common Variations:

- Max profit without exceeding capacity
- Subset sum decisions (True/False)
- Count of valid combinations

🔥 FAANG Examples:

- Leetcode 416 - Partition Equal Subset Sum
- Leetcode 494 - Target Sum
- Leetcode 377 - Combination Sum IV
- GFG - 0/1 Knapsack
- GFG - Count of Subsets with Given Sum

42. 🧠 DP – Subset / Boolean DP

📌 Track whether **a condition is possible** (like target sum, balanced partition).

🎯 You don't care **how many ways**, just whether you can reach a certain **state**.

✨ Think of it like: "Can I reach sum **s** using any subset of the first **i** elements?"

🎯 **Why it matters:**

- Useful in **decision logic**, where you care about **possibility** not quantity.

💡 **Hot in Meta** for logic-heavy questions.

🔥 FAANG Examples:

- Leetcode 698 - Partition to K Equal Sum Subsets
 - Leetcode 139 - Word Break
 - GFG - Equal Sum Partition
 - Leetcode 1049 - Last Stone Weight II
-

43. 🧠 DP – Unbounded Knapsack

📌 Like 0/1 knapsack, **but you can reuse items unlimited times**.

🎯 Useful in coin change, cutting rod, unbounded combinations

✨ You build solutions **bottom-up**, trying all possibilities with repetition allowed.

🎯 **Why it matters:**

- Teaches **combination generation** and **DP loops optimization**.

💡 **Popular at Google**, often hidden in coin change-like problems.

🔥 FAANG Examples:

- Leetcode 322 - Coin Change
 - Leetcode 518 - Coin Change II
 - GFG - Rod Cutting Problem
 - Leetcode 70 - Climbing Stairs
-

44. 🧠 DP – Minimum/Maximum Cost Path

📌 You want to **minimize or maximize a value** while reaching a destination (like grid bottom-right).

🎯 These use **2D bottom-up DP** tracking the best cost to each cell.

✨ Frequently applied in grid movement, edit distance, merge cost

🎯 **Why it matters:**

- Teaches **DP table filling**, great for matrix-based systems.

💡 **Asked frequently at Apple and Microsoft** for path optimization logic

🔥 FAANG Examples:

- [Leetcode 64 - Minimum Path Sum](#)
- [Leetcode 931 - Minimum Falling Path Sum](#)
- [Leetcode 72 - Edit Distance](#)
- [Leetcode 115 - Distinct Subsequences](#)

45. 🧠 DP – Count All Possibilities

📌 You're asked: "How many ways to reach X?" → Count all **valid paths/ways** to reach a state

🎯 Often uses recurrence like $dp[i] = dp[i-1] + dp[i-2] + \dots$

✨ Great for permutations, dice rolls, stairs, encodings

🎯 **Why it matters:**

- Teaches **DP recurrence patterns and bottom-up building**.

💡 **Loved by Google** and **Adobe** for classic dynamic counting questions

🔥 FAANG Examples:

- [Leetcode 70 - Climbing Stairs](#)
- [Leetcode 91 - Decode Ways](#)
- [Leetcode 509 - Fibonacci Number](#)
- [Leetcode 62 - Unique Paths](#)
- [Leetcode 377 - Combination Sum IV](#)

46. 🧠 DP – Longest Common Subsequence (LCS)

📌 You want the longest sequence **appearing in both strings (not necessarily contiguous)**

🎯 The subproblem is: "If last characters match, extend +1; else take max of exclude cases."

✨ This base logic drives multiple other patterns like edit distance, diff, subsequence match

🎯 **Why it matters:**

- Powers many **diff/merge tools, version control, edit distance problems**.
- 💡 **Hot in Amazon and Dropbox** for string diffing.

🔥 FAANG Examples:

- [Leetcode 1143 - Longest Common Subsequence](#)
 - [Leetcode 72 - Edit Distance](#)
 - [Leetcode 115 - Distinct Subsequences](#)
 - [Leetcode 583 - Delete Operations for Two Strings](#)
-

47. 🧠 DP – Longest Palindromic Subsequence

📌 Use LCS on the string vs its reverse to find max-length palindrome subsequence

🎯 Classic use of 2D DP matrix comparing forward vs reverse

✨ Interviewers often ask this to test **transformation-based thinking**

🎯 **Why it matters:**

- **Loved by Microsoft**, tests string transformation and DP symmetry.
- Appears in string transformation, DNA matching, and encryption rounds.

🔥 FAANG Examples:

- [Leetcode 516 - Longest Palindromic Subsequence](#)
 - [Leetcode 1312 - Minimum Insertion Steps to Make a String Palindrome](#)
 - [GFG - Count Palindromic Subsequences](#)
-

48. 🧠 DP – Longest Increasing Subsequence (LIS)

📌 Track the **longest increasing order** of numbers in a sequence

🎯 Brute force is $O(n^2)$ with DP; optimized with binary search to $O(n \log n)$

✨ Shows up in stock, buildings, box stacking, patience sorting

🎯 **Why it matters:**

- Common in **stock, building, envelope stacking, patience sorting** problems.

💡 **Very popular at Google** for optimization + sorting rounds.

🔥 FAANG Examples:

- [Leetcode 300 - Longest Increasing Subsequence](#)
- [Leetcode 673 - Number of LIS](#)
- [Leetcode 354 - Russian Doll Envelopes](#)
- [Leetcode 1186 - Maximum Subarray Sum with One Deletion](#)

🧠 SECTION 5B: DYNAMIC PROGRAMMING PATTERNS (Part 2 – Palindrome, Bitmask, Partition, Digit DP)

49. 🧠 Palindrome Partitioning (Recursive + DP)

📌 **Partition the string** such that every substring is a palindrome, and minimize cuts or count partitions.

🎯 Core Idea: Use **recursion with memorization**, and check palindromes using a 2D DP table or substring expansion.

💡 Why it matters: Tests string pre-processing + multi-stage recursion with caching.

🎯 **Why it matters:**

- Combines **string + DP + memorization + expansion**

💡 A **hot favorite at Google and Adobe**, appears in top DP playlists

🔄 Variants:

- Min cuts needed
- List all valid partitions
- Count of partitions

🔥 FAANG Examples:

- [Leetcode 131 - Palindrome Partitioning](#)
- [Leetcode 132 - Palindrome Partitioning II](#)
- [GFG - Palindrome Partitioning \(Memoization\)](#)

50. 🧠 Digit DP (Number-based DP)

📌 Count how many numbers **satisfy a property**, from 0 to N.

🎯 Core Idea: Build digits **one-by-one**, tracking constraints like tight-bound, leading-zero, used-digits.

💡 Why it matters: This is the backbone of Google-level number system problems.

🎯 **Why it matters:**

- Powerful in **Google bar raiser rounds**
- Tracks **digit-by-digit state transitions** (tight, leading zero, mod conditions)

 Variants:

- Count of numbers divisible by X
- Count of beautiful numbers
- Count of valid digit combinations under constraints

 FAANG Examples:

- [Leetcode 233 - Number of Digit One](#)
 - [Leetcode 902 - Numbers At Most N Given Digit Set](#)
 - [Leetcode 600 - Non-negative Integers without Consecutive Ones](#)
 - [Leetcode 1012 - Numbers With Repeated Digits](#)
-

51. Bitmask DP (State Compression)

 Use a **bitmask to track used elements** in the state (usually in subset DP).

 Core Idea: Represent current state of selections as a bitmask.

 Why it matters: Solves exponential problems efficiently using smart memorization.

 **Why it matters:**

- Useful in **traveling salesman, job assignment, and subset optimization**
-  **Used heavily by Meta and Google** in hard coding rounds.

 Variants:

- TSP (Travelling Salesman)
- Partitioning subsets
- Perfect matchings in graphs

 FAANG Examples:

- [Leetcode 698 - Partition to K Equal Sum Subsets](#)

- [Leetcode 847 - Shortest Path Visiting All Nodes](#)
 - [Leetcode 1125 - Smallest Sufficient Team](#)
 - [Leetcode 691 - Stickers to Spell Word](#)
-

52. 🧠 DP with State + Position

📌 Store more than one parameter in your DP state — e.g., `dp[i][sum][count]`

🎯 Core Idea: State depends on **position + choices made so far**

💡 Why it matters: Helps break down complex constraints step-by-step

🎯 **Why it matters:**

- Excellent for **multi-variable tracking**, frequently used in **stock market problems**.

💡 **Popular at Apple and Goldman Sachs.**

🔄 Variants:

- Subset sum with size limit
- Choosing k elements with exact sum
- Buying/selling stocks with cooldowns/limits

🔥 FAANG Examples:

- [Leetcode 198 - House Robber](#)
 - [Leetcode 213 - House Robber II](#)
 - [Leetcode 123 - Best Time to Buy and Sell Stock III](#)
 - [Leetcode 309 - Best Time with Cooldown](#)
-

53. 🧠 DP on Strings – Pattern Matching

📌 Solve problems where you compare a pattern to a string, like regex, wildcard, subsequences

🎯 Core Idea: $DP[i][j]$ = match of first i of pattern and first j of text

💡 Why it matters: Combines string theory + conditional DP

🎯 Why it matters:

- Common in **text editors, regex engines, compiler parsing**
- 💡 **Facebook and Dropbox** love asking these

🔄 Variants:

- Regular expressions
- Wildcards
- Subsequence validation

🔥 FAANG Examples:

- [Leetcode 10 - Regular Expression Matching](#)
 - [Leetcode 44 - Wildcard Matching](#)
 - [Leetcode 115 - Distinct Subsequences](#)
 - [Leetcode 392 - Is Subsequence](#)
-

54. 🧠 Matrix Chain Multiplication (Interval DP)

📌 Find the optimal way to group/multiply a series of operations to minimize cost

🎯 Core Idea: Break the sequence into intervals — try every valid split

💡 Why it matters: Good test of DP table building, often appears disguised

🎯 Why it matters:

- Good for **editor tools, parsers, dynamic grouping**
- 💡 Asked in **Google + Oracle** for recursive interval breakdown.

🔄 Variants:

- Minimum cost of grouping
- DP over parentheses
- Balloon burst variations

🔥 FAANG Examples:

- [Leetcode 312 - Burst Balloons](#)

- [GFG - Matrix Chain Multiplication](#)
 - [Leetcode 1000 - Minimum Cost to Merge Stones](#)
-

55. 🧠 Palindromic Substrings (Expand + DP)

📌 Count or extract all palindromic substrings in a string

🎯 Core Idea: Expand from center or use a 2D DP table to track palindrome ranges

💡 Why it matters: Palindrome-based questions are **recurring FAANG favorites**

🎯 **Why it matters:**

- Appears in **text analysis, DNA search, cybersecurity checks**
 - 💡 A favorite in **Amazon + Adobe** mid-level DP rounds

🔄 Variants:

- Longest palindromic substring
- Count of palindromic substrings
- Palindrome deletions

🔥 FAANG Examples:

- [Leetcode 647 - Palindromic Substrings](#)
 - [Leetcode 5 - Longest Palindromic Substring](#)
 - [Leetcode 132 - Palindrome Partitioning II](#)
 - [Leetcode 516 - Longest Palindromic Subsequence](#)
-

56. 🧠 DP + Greedy Hybrid

📌 Sometimes, you **preprocess greedily**, then use DP to evaluate results

🎯 Core Idea: Greedy gives you a candidate, DP helps validate/minimize/maximize

💡 Why it matters: Real-world problems often **combine** strategies

🎯 **Why it matters:**

- Real-world problems often blend multiple strategies
 - 💡 Common in **Uber + Atlassian** scheduling questions.

Variants:

- Scheduling with DP
- Choosing leaders
- Job sequencing

FAANG Examples:

- [Leetcode 140 - Word Break II](#)
- [Leetcode 188 - Best Time to Buy and Sell Stock IV](#)
- [Leetcode 1326 - Minimum Number of Taps](#)
- [Leetcode 1235 - Maximum Profit in Job Scheduling](#)

SECTION 6: TREE TRAVERSAL & TREE-BASED PATTERNS

57. Tree Traversals – Inorder (Left → Root → Right)

 Traverse left subtree, root, then right subtree.

 **Why it matters:**

- Used to retrieve **sorted order in BST**
- Base for constructing and comparing trees
- Important for **BST problems in Google, Microsoft**
- Helps solve **rank-based queries** in trees (like Kth smallest)

 Examples:

- [Leetcode 94 - Binary Tree Inorder Traversal](#)
 - [Leetcode 230 - Kth Smallest in BST](#)
-

58. Tree Traversals – Preorder (Root → Left → Right)

 Visit root first, then traverse left and right subtrees.

 **Why it matters:**

- Used in **tree copying, serialization**, and expression trees
- Common in building trees from traversal arrays (e.g. preorder + inorder)

 Examples:

- [Leetcode 144 - Preorder Traversal](#)
 - [Leetcode 105 - Construct Tree from Preorder and Inorder](#)
-

57. Tree Traversals – Postorder (Left → Right → Root)

 Traverse children first, then visit the root.

 **Why it matters:**

- Helps in **deleting nodes, memory cleanup, and computing tree height**

- Common in post-processing subtrees like for sum, balance, or evaluation
- Common in **Amazon, Adobe, Microsoft**

🔥 Examples:

- [Leetcode 145 - Postorder Traversal](#)
 - [GFG - Delete Tree](#)
-

60. 🧠 Tree Traversals – Level Order (Breadth-First Search)

📌 Visit tree level-by-level using a queue.

🎯 **Why it matters:**

- Used in **hierarchical grouping, depth analysis**, and systems like comment threads
- Forms basis for **zigzag, vertical, and bottom-up traversals**

🔥 Examples:

- [Leetcode 102 - Binary Tree Level Order Traversal](#)
 - [Leetcode 103 - Zigzag Level Order](#)
 - [Leetcode 107 - Bottom-Up Level Order](#)
-

61. 🧠 Binary Tree Diameter

📌 At each node: $\max(\text{leftHeight} + \text{rightHeight})$

🎯 **Why it matters:**

- Teaches recursive subtree analysis
- Helps analyze **longest path** through any two nodes
- Reinforces **post-order recursion** and subtree height tracking

🔥 Example:

- [Leetcode 543 - Diameter of Binary Tree](#)
-

62. 🧠 Lowest Common Ancestor (LCA)

📌 Find the **lowest node** that has both target nodes in its subtrees.

🎯 **Why it matters:**

- Basis of **hierarchy resolution, folder trees, class inheritance**
- Binary Tree: requires DFS logic; BST: use value comparisons for efficient path
- Recursively useful in graphs and **file system design**

🔄 Variants:

- LCA in binary tree
- LCA in BST (use value comparisons)

🔥 Examples:

- [Leetcode 236 - LCA of Binary Tree](#)
 - [Leetcode 235 - LCA of BST](#)
-

63. 🧠 Symmetric Tree / Mirror Check

📌 Check if left subtree is mirror of right subtree.

🎯 **Why it matters:**

- Tests **structural recursion**
- Asked for visual layouts and **AI tree reflections**
- Requires recursive mirroring logic: `left.left == right.right`

🔥 Example:

- [Leetcode 101 - Symmetric Tree](#)
-

64. 🧠 Invert Binary Tree

📌 Swap left/right of every node.

🎯 **Why it matters:**

- Easy question but tests recursive design
- Common in **LinkedIn + Uber** warm-up rounds

🔥 Example:

- [Leetcode 226 - Invert Binary Tree](#)
-

65. 🧠 Path Sum Variants

📌 Use DFS to compute sum along root-to-leaf or all paths.

🎯 Why it matters:

- Enhances understanding of **root-to-leaf recursion, pruning, and sum tracking**
- Appears in **Amazon, Apple**, and system dependency evaluation
- Tests DFS and pruning logic

🔄 Variants:

- Path exists
- Return all paths
- Count total paths

🔥 Examples:

- [Leetcode 112 - Path Sum](#)
 - [Leetcode 113 - Path Sum II](#)
 - [Leetcode 437 - Path Sum III](#)
-

66. 🧠 Serialize and Deserialize Binary Tree

📌 Convert a tree to string (serialization) and back (deserialization).

🎯 Why it matters:

- Basis for **database/tree storage, transmission protocols**, etc.
- Uses preorder or level order with null markers

🔥 Example:

- [Leetcode 297 - Serialize and Deserialize Binary Tree](#)
-

67. 🧠 Construct Tree from Traversals

📌 Rebuild tree from preorder/inorder/postorder combos.

🎯 **Why it matters:**

- Tests **recursion + slicing arrays + index maps**
- Popular in **Dropbox, Google**, and tree construction logic

🔥 **Examples:**

- [Leetcode 105 - Preorder + Inorder](#)
- [Leetcode 106 - Inorder + Postorder](#)

SECTION 7: GRAPHS & BACKTRACKING

68. Depth-First Search (DFS)

 Traverse deeply down one path before backtracking.

 **Why it matters:**

- Used for **component detection, maze traversal, graph coloring, cycle detection.**
- Highly recursive, often a **base layer for tree/graph problems.**
- Loved in **Facebook, Google** for recursion logic.

 Variants:

- DFS on grids
- DFS on adjacency list
- Recursive vs iterative

 Examples:

- [Leetcode 200 - Number of Islands](#)
- [Leetcode 695 - Max Area of Island](#)
- [Leetcode 130 - Surrounded Regions](#)
- [Leetcode 547 - Number of Provinces](#)
- [Leetcode 417 - Pacific Atlantic Water Flow](#)

69. Breadth-First Search (BFS)

 Explore all neighbors before going deeper (level-by-level).

 **Why it matters:**

- Essential for **shortest path, multi-source expansion, and minimum moves** problems.
- Used in **Google and Uber** to simulate flood fills, spreading, routing.

 Variants:

- BFS in matrix
- BFS on graphs
- Multi-source BFS

🔥 Examples:

- [Leetcode 994 - Rotting Oranges](#)
 - [Leetcode 1091 - Shortest Path in Binary Matrix](#)
 - [Leetcode 542 - 01 Matrix](#)
 - [Leetcode 127 - Word Ladder](#)
 - [Leetcode 752 - Open the Lock](#)
-

70. 🧠 Topological Sort

📌 Linear ordering of DAG (Directed Acyclic Graph) nodes.

🎯 **Why it matters:**

- Key in **course scheduling, task dependency resolution, compilation order**.
- Appears in **Uber, Amazon, Google** task orchestration problems.

🔄 Variants:

- DFS-based
- Kahn's Algorithm (BFS)
- Cycle detection in DAG

🔥 Examples:

- [Leetcode 207 - Course Schedule](#)
 - [Leetcode 210 - Course Schedule II](#)
 - [Leetcode 269 - Alien Dictionary](#)
 - [GFG - Detect Cycle in Directed Graph](#)
-

71. 🧠 Union Find (Disjoint Set)

📌 Track connected components efficiently using **parent pointers + path compression**.

🎯 **Why it matters:**

- Used in **clustering, dynamic connectivity, network queries, account merging**.
- Found in **Dropbox, Meta, and AWS backend rounds**.

🔄 Variants:

- Cycle detection
- Connected components
- Merge groups/accounts

🔥 Examples:

- [Leetcode 684 - Redundant Connection](#)
 - [Leetcode 1319 - Make Network Connected](#)
 - [Leetcode 721 - Accounts Merge](#)
 - [Leetcode 959 - Regions Cut by Slashes](#)
-

72. 🧠 Backtracking (Combinations/Permutations)

📌 Recursively try all possible decisions and **backtrack when invalid**.

🎯 **Why it matters:**

- Core to **constraint satisfaction, permutations, combinatorics**
- Shows ability to design **recursion trees + pruning** logic — often tested at **Meta, Apple, Google**.

🔄 Variants:

- All combinations
- Unique permutations
- Skip duplicates

🔥 Examples:

- [Leetcode 46 - Permutations](#)
 - [Leetcode 47 - Permutations II](#)
 - [Leetcode 77 - Combinations](#)
 - [Leetcode 39 - Combination Sum](#)
 - [Leetcode 40 - Combination Sum II](#)
-

73. 🧠 Backtracking (Constraint-based)

📌 Explore state space while ensuring **each move satisfies constraints**.

🎯 **Why it matters:**

- Tests **recursion, constraint checking, pruning**
- Used in **Sudoku, chess puzzles, N-Queens** — very common in **Google + Adobe**

🔄 Variants:

- Sudoku solving
- N-Queens placement
- Word paths

🔥 Examples:

- [Leetcode 51 - N-Queens](#)
 - [Leetcode 37 - Sudoku Solver](#)
 - [Leetcode 212 - Word Search II](#)
 - [Leetcode 79 - Word Search](#)
-

74. 🧠 Graph Cycle Detection (DFS & Union-Find)

📌 Check if the graph contains any cycle.

🎯 **Why it matters:**

- Appears in **dependency resolution, build order, and network analysis**.
- Combines **DFS recursion stack** or **Union-Find sets**.

 Variants:

- Directed graph → DFS + RecStack
- Undirected → Union-Find
- Detect cycle origin

 Examples:

- [Leetcode 207 - Course Schedule](#)
 - [Leetcode 261 - Graph Valid Tree](#)
 - [Leetcode 785 - Is Graph Bipartite?](#)
 - [GFG - Detect Cycle in Undirected Graph](#)
-

75. Shortest Path Algorithms

 Compute shortest distance between nodes.

 Why it matters:

- Common in **routing, flight optimization, map systems**
- FAANG asks this to test **weight handling + graph exploration**

 Variants:

- BFS for unweighted
- Dijkstra for weighted
- Bellman-Ford for negative weights

 Examples:

- [Leetcode 743 - Network Delay Time](#)
 - [Leetcode 1631 - Path with Minimum Effort](#)
 - [Leetcode 787 - Cheapest Flights Within K Stops](#)
 - [Leetcode 778 - Swim in Rising Water](#)
-

76. Word Ladder Pattern (Graph Implicitly Represented)

📌 Words are nodes, edges connect if one letter differs.

🎯 **Why it matters:**

- Tests ability to **model graphs from constraints**, not explicit edges
- Loved by **Google** and **Facebook** for testing **transformational graph modeling**.

🔄 Variants:

- Shortest transformation
- Return all transformation paths

🔥 Examples:

- [Leetcode 127 - Word Ladder](#)
 - [Leetcode 126 - Word Ladder II](#)
-

77. 🧠 Maze/Grid BFS/DFS Hybrid

📌 Grid traversal problems with constraints like walls, teleports, steps.

🎯 **Why it matters:**

- Evaluates ability to combine **graph traversal** with **grid simulation**
- Common in **game simulation, robotics, and matrix navigation**

🔄 Variants:

- Find exit
- Find shortest time
- All reachable cells

🔥 Examples:

- [Leetcode 490 - The Maze](#)
- [Leetcode 505 - The Maze II](#)
- [Leetcode 499 - The Maze III](#)
- [Leetcode 329 - Longest Increasing Path in Matrix](#)

⚡ SECTION 8: GREEDY + BIT MANIPULATION + MATH

These patterns are **high-yield** — fast, tricky, and heavily used in **Google, Uber, and finance-based roles**.

78. 🧠 Greedy – Interval Scheduling

📌 Choose non-overlapping intervals to **maximize count or value**

🎯 Core Idea: Sort intervals (by end time or start time), then **greedily pick the optimal next**

💡 Why it matters: Real-world in meetings, jobs, ads, and timeline events

🔄 Variants:

- Max non-overlapping
- Min removals
- Room allocation

🔥 FAANG Examples:

- [Leetcode 435 - Non-overlapping Intervals](#)
 - [Leetcode 452 - Minimum Number of Arrows to Burst Balloons](#)
 - [Leetcode 253 - Meeting Rooms II](#)
 - [GFG - Activity Selection](#)
-

79. 🧠 Greedy – Jump Game Pattern

📌 Move through an array using **minimum jumps or farthest reach**

🎯 Core Idea: Always **choose the furthest you can go** from the current jump

💡 Why it matters: Models decisions under limited steps or actions

🔄 Variants:

- Can you reach the end?
- Min jumps to reach end

- Max range covered

🔥 FAANG Examples:

- [Leetcode 55 - Jump Game](#)
 - [Leetcode 45 - Jump Game II](#)
 - [Leetcode 1345 - Jump Game IV](#)
-

80. 🧠 Greedy – Gas Station Loop

📌 Circular problem where you gain/lose value per index

🎯 Core Idea: Use **cumulative gain/loss** and restart only at points of deficit

💡 Why it matters: Combines array math + greedy checkpointing

🔄 Variants:

- Circular route with conditions
- Total supply vs demand logic

🔥 FAANG Examples:

- [Leetcode 134 - Gas Station](#)
 - [Leetcode 918 - Maximum Sum Circular Subarray](#)
-

81. 🧠 Greedy – Monotonic Stack (Span Problems)

📌 Use a stack to **maintain order** (increasing/decreasing)

🎯 Core Idea: Efficiently resolve next/previous greater/smaller in $O(n)$

💡 Why it matters: Used in stock prices, histograms, spans, temperatures

🔄 Variants:

- Stock span
- Histogram max area
- Nearest smaller elements

🔥 FAANG Examples:

- [Leetcode 84 - Largest Rectangle in Histogram](#)

- [Leetcode 739 - Daily Temperatures](#)
 - [Leetcode 901 - Online Stock Span](#)
-

82. 🧠 Greedy – Change Making (Coin Problem)

📌 Give change using **least number of coins**

🎯 Core Idea: Use largest coins first (if denominations are canonical)

💡 Why it matters: Practical, and shows where greedy **fails**

🔄 Variants:

- Fewest coins (Greedy vs DP)
- Infinite supply problems
- Currency systems

🔥 FAANG Examples:

- [Leetcode 322 - Coin Change](#) (DP, greedy fails)
 - [GFG - Minimum Coins to Make Sum](#)
 - [Leetcode 135 - Candy Distribution](#)
-

83. 🧠 Bit Manipulation – Single Number Variants

📌 Use XOR and bitwise ops to **find unique/non-repeating elements**

🎯 Core Idea: XOR cancels duplicates: $a \oplus a = 0$

💡 Why it matters: Optimizes problems needing **O(1) space** without hashmaps

🔄 Variants:

- 1 unique among duplicates
- 2 unique numbers
- Bit counting

🔥 FAANG Examples:

- [Leetcode 136 - Single Number](#)
- [Leetcode 137 - Single Number II](#)

- [Leetcode 260 - Single Number III](#)
 - [Leetcode 201 - Bitwise AND of Numbers Range](#)
-

84. 🧠 Bit Manipulation – Subsets by Masking

📌 Generate subsets using a **bitmask from 0 to $(2^n - 1)$**

🎯 Core Idea: Mask $1 \ll i$ to decide inclusion of elements

💡 Why it matters: Clean alternative to backtracking for powerset

🔄 Variants:

- All subsets
- Subsets summing to X
- Bitmask DP

🔥 FAANG Examples:

- [Leetcode 78 - Subsets](#)
 - [Leetcode 90 - Subsets II](#)
 - [Leetcode 1986 - Minimum Number of Work Sessions](#)
-

85. 🧠 Math – GCD / LCM Utilities

📌 Use **Euclidean algorithm** for GCD and derive LCM

🎯 Core Idea: $\text{GCD}(a, b) = \text{GCD}(b, a \% b)$

💡 Why it matters: Appears in divisibility, synchronization, ratios

🔄 Variants:

- GCD of pairs/arrays
- LCM alignments
- Fractions

🔥 FAANG Examples:

- [Leetcode 1071 - GCD of Strings](#)
- [Leetcode 149 - Max Points on a Line \(uses slope simplification via GCD\)](#)

- [GFG - GCD of Array](#)
-

86. 🧠 Math – Sieve of Eratosthenes

- 📌 Precompute primes up to N efficiently
- 🎯 Core Idea: Mark multiples of primes starting from 2
- 💡 Why it matters: Crucial in number theory problems
- 🔄 Variants:
 - Count primes
 - Prime sum pairs
 - Goldbach conjecture-style
- 🔥 FAANG Examples:
 - [Leetcode 204 - Count Primes](#)
 - [Leetcode 1175 - Prime Arrangements](#)
 - [GFG - Sieve Variants](#)

SECTION 9: ADVANCED STRING MANIPULATION + TRIES + SEGMENT TREES

87. String Sliding Window with Frequency Maps

 Move a window over a string while tracking **character counts dynamically**

 Core Idea: Expand window, update map; if invalid, contract

 Why it matters: Interviewers love this for parsing and substring challenges

 **Hot at:** Google, Atlassian, Twitter

 Variants:

- Smallest substring with all chars
- Longest substring with at most K
- Repeating/unique characters

 Examples:

- [Leetcode 76 - Minimum Window Substring](#)
 - [Leetcode 3 - Longest Substring Without Repeating Characters](#)
 - [Leetcode 159 - Longest Substring with At Most Two Distinct Characters](#)
-

88. Two-Pass String Processing

 First pass for counting/indexing, second for building/modifying

 Core Idea: Pass 1 → map/index, Pass 2 → validate/construct

 Why it matters: Often used in parsing, encodings, decoding

 **Loved by:** Meta, Apple, Microsoft

 Variants:

- Reformatting
- Removing invalid characters
- Compress/expand strings

 Examples:

- [Leetcode 443 - String Compression](#)
 - [Leetcode 1249 - Minimum Remove to Make Valid Parentheses](#)
 - [Leetcode 5 - Longest Palindromic Substring](#)
-

89. 🧠 String Builder Optimization

📌 When frequent concatenation is needed, use buffer instead

🎯 Core: Avoid expensive immutable `+` operations

💡 Why: Performance boost + cleaner logic

🔍 **Loved by:** Amazon, Adobe, Expedia

🔄 Variants:

- Run-length encoding
- HTML/minifier-style parsing
- Decoding instructions

🔥 Examples:

- [Leetcode 394 - Decode String](#)
 - [Leetcode 151 - Reverse Words in a String](#)
 - [Leetcode 6 - Zigzag Conversion](#)
-

90. 🧠 KMP (Knuth-Morris-Pratt) Pattern Matching

📌 Efficient substring search using a **prefix table**

🎯 Core: Build LPS array to avoid re-checking characters

💡 Why: Massive optimization for long pattern matching

🔍 **Loved by:** Google, Oracle, Nvidia

🔄 Variants:

- Substring search
- Repeating patterns
- String rotations

🔥 Examples:

- Leetcode 28 - Find the Index of First Occurrence in a String
 - Leetcode 459 - Repeated Substring Pattern
 - GFG - Pattern Matching with KMP
-

91. 🧠 Rabin-Karp + Rolling Hash

📌 Use rolling hash for fast substring comparisons

🎯 Core: Convert strings to numbers (with base+mod)

💡 Why: Powerful in detecting **duplicates, plagiarism, large-scale logs**

🔍 **Hot at:** Google, Bloomberg, ByteDance

🔄 Variants:

- Duplicate substrings
- Pattern match with rolling scan
- Detect changes in streams

🔥 Examples:

- Leetcode 1044 - Longest Duplicate Substring
- Leetcode 187 - Repeated DNA Sequences

TRIE (Prefix Tree) PATTERNS

92. Basic Trie Construction & Search

 Store words/strings in a prefix tree

 Core: Each level = one char; node marks end

 Why: Blazing-fast prefix and exact match searches

 **Loved by:** Google, Amazon, Spotify

 Variants:

- Autocomplete
- Search suggestions
- StartsWith vs full match

 Examples:

- [Leetcode 208 - Implement Trie \(Prefix Tree\)](#)
 - [Leetcode 211 - Add and Search Word](#)
 - [Leetcode 648 - Replace Words](#)
-

93. Trie with DFS for Word Search

 Use Trie to hold words, then DFS on board to match

 Core: Trie node → DFS state combo

 Why: Combines Trie + Graph DFS for **efficient multi-word search** on 2D matrices.

 **Hot at:** Meta, ByteDance, Google

 Variants:

- Boggle solver
- Word search on board
- Multi-word prefix tree traversal

 Examples:

- [Leetcode 212 - Word Search II](#)
 - [Leetcode 720 - Longest Word in Dictionary](#)
 - [Leetcode 1268 - Search Suggestions System](#)
-

94. 🧠 Bitwise Trie (Binary Trie)

📌 Store numbers in trie using **binary representation**

🎯 Core: Traverse bit-by-bit for max/min XOR

💡 Why: Crucial in bitwise problems where numbers behave like strings

★ **Why it matters:** Solves complex bitwise questions in **$O(1)$ traversal time**, often seen in XOR-based optimization.

🔍 **Loved by:** Google, Nvidia, Qualcomm

🔄 Variants:

- Max XOR pair
- Range XOR
- Subarray XORs

🔥 Examples:

- [Leetcode 421 - Maximum XOR of Two Numbers](#)
- [Leetcode 677 - Map Sum Pairs](#)
- [GFG - Max XOR with Trie](#)



SEGMENT TREE PATTERNS

95. 🧠 Segment Tree for Range Queries

📌 A powerful data structure that **preprocesses an array into a tree** to answer range-based queries (like sum, min, max, gcd) in **$\log(n)$** time.

🎯 Core Idea:

Divide the array into segments. Each node in the tree stores a **precomputed value** for its range.

When you query, the tree smartly combines answers from relevant nodes — super fast and efficient.

🌟 Why it matters:

When you have frequent **range queries + real-time updates**, brute force becomes too slow.

Segment Trees shine in problems like:

- 📈 **Stock price tracking**
- 📊 **Live leaderboards & analytics**
- 🕒 **Time-range calendars**
- 📦 **Subarray statistics with fast updates**

🔍 **Hot in:** Amazon, Google, and widely used in Competitive Programming (Codeforces, AtCoder)

🔄 Common Variants:

- Point update + range sum/min/max
- Range update with lazy propagation
- Dynamic segment trees for sparse arrays

🔥 Example Problems:

- Leetcode 307 - Range Sum Query - Mutable
- Leetcode 732 - My Calendar III
- GFG - Range Minimum Query (RMQ)

96. 🧠 Lazy Propagation in Segment Trees

📌 Delay updates to reduce operations until actually needed

🎯 Core: Track pending updates in separate structure

💡 Why: Boosts performance when multiple updates are pending

🌟 **Why it matters:** Optimizes **batch updates, toggles, and window scheduling systems**.

🔍 **Loved by:** Top CP platforms, Google, Codeforces

🔄 Variants:

- Range updates
- Flip/Increment elements
- Segment toggles

🔥 Examples:

- [GFG - Lazy Propagation in Segment Tree](#)
- [Codeforces - Multiples and Power of 2 \(Segment Tree + Lazy\)](#)



SECTION 10: MATRIX, GEOMETRY & SIMULATION PATTERNS

97. 🧠 Matrix DFS/BFS Traversal

📌 Traverse 2D grids using **DFS (recursion)** or **BFS (queue)** in 4 or 8 directions.

🎯 **Core Idea:** Use direction vectors (`dx` , `dy`) to explore adjacent cells. Mark visited to avoid cycles.

💡 **Why it matters:** It's the **foundation of all grid-based problems** — from islands to area filling.

🔄 Variants:

- Island counting
- Connected region analysis
- Shortest path on grid

🔥 Examples:

- `Leetcode 200 - Number of Islands`
 - `Leetcode 994 - Rotting Oranges`
 - `Leetcode 542 - 01 Matrix`
-

98. 🧠 Spiral Matrix Traversal

📌 Simulate movement in a **spiral pattern** (clockwise or counterclockwise) using boundaries.

🎯 **Core Idea:** Maintain `top` , `bottom` , `left` , `right` boundaries and shrink after each layer.

💡 **Why it matters:** Common interview simulation problem that tests boundary updates.

🔄 Variants:

- Print matrix spirally
- Fill values spirally

🔥 Examples:

- [Leetcode 54 - Spiral Matrix](#)
 - [Leetcode 59 - Spiral Matrix II](#)
-

99. 🧠 Diagonal Matrix Traversal

📌 Traverse diagonals of a 2D matrix starting from top-left.

🎯 **Core Idea:** Simulate alternating up-right and down-left directions; track diagonal levels.

💡 **Why it matters:** Appears in **data compression, matrix output formatting, and zigzag paths**.

🔄 Variants:

- Zigzag diagonals
- Group diagonals by sum of (row + col)

🔥 Examples:

- [Leetcode 498 - Diagonal Traverse](#)
 - [GFG - Print Diagonals of Matrix](#)
-

100. 🧠 Rotate Matrix In-place

📌 Rotate a square matrix by 90°, 180°, or 270° without using extra space.

🎯 **Core Idea:** Use **transpose + reverse** or **4-way swap** in layers.

💡 **Why it matters:** Tests **in-place thinking, memory optimization, and spatial logic**.

🔄 Variants:

- 180° or 270° rotation
- Reverse diagonals

🔥 Examples:

- [Leetcode 48 - Rotate Image](#)

- [Leetcode 867 - Transpose Matrix](#)
-

101. 🧠 Flood Fill / Region Filling

📌 Fill all connected cells of the same value starting from a source.

🎯 **Core Idea:** Recursively visit same-color neighbors and update their value.

💡 **Why it matters:** Mimics **image painting, map coloring, and graph fill logic.**

🔄 Variants:

- Region capture
- Bounded fill

🔥 Examples:

- [Leetcode 733 - Flood Fill](#)
 - [Leetcode 130 - Surrounded Regions](#)
-

102. 🧠 Geometry – Area, Distance, Collinearity

📌 Solve geometric problems using **coordinate formulas.**

🎯 **Core Idea:** Use slope for collinearity, area formulas for shapes, and Euclidean distance.

💡 **Why it matters:** Appears in **map engines, drone paths, and spatial reasoning.**

🔄 Variants:

- Closest pair of points
- Valid square
- Intersecting rectangles

🔥 Examples:

- [Leetcode 149 - Max Points on a Line](#)
 - [Leetcode 223 - Rectangle Area](#)
 - [Leetcode 593 - Valid Square](#)
-

103. 🧠 Math – Modular Arithmetic

📌 Perform calculations under a **modulo** to handle overflow and cyclic logic.

🎯 **Core Idea:** Use modular properties to safely compute sums, products, and powers.

💡 **Why it matters:** Essential for **hashing, cryptography, and circular systems**.

🔄 Variants:

- Modulo inverse
- Modulo exponentiation
- Time wrap-around

🔥 Examples:

- [Leetcode 535 - Encode and Decode TinyURL](#)
 - [Leetcode 1491 - Average Salary Excluding Min and Max](#)
 - [Leetcode 1041 - Robot Bounded in Circle](#)
-

104. 🧠 Fast Power / Binary Exponentiation

📌 Compute a^b in $\log(b)$ time using divide-and-conquer.

🎯 **Core Idea:** Recurse with $b/2$, multiply result. If b is odd, multiply a one more time.

💡 **Why it matters:** Core to **RSA, cryptography, and time-constrained math problems**.

🔄 Variants:

- Modular exponentiation
- Exponentiation with large b

🔥 Examples:

- [Leetcode 50 - Pow\(x, n\)](#)
 - [Leetcode 372 - Super Pow](#)
-

105. 🧠 Combinatorics – nCr / Pascal's Triangle

📌 Count combinations using math formula or dynamic triangle.

🎯 **Core Idea:** $nCr = n! / (r!(n-r)!)$ or recursively build Pascal's triangle row by row.

💡 **Why it matters:** Powers problems in **probability, grid paths, and combinations**.

🔄 Variants:

- Grid path counting
- Unique selections
- Triangular DP

🔥 Examples:

- [Leetcode 118 - Pascal's Triangle](#)
 - [Leetcode 119 - Pascal's Triangle II](#)
 - [Leetcode 62 - Unique Paths](#)
-

106. 🧠 Clock / Circular Array Problems

📌 Handle **wrap-around behavior** in circular structures.

🎯 **Core Idea:** Use modulo `%` and handle edge wrap logic manually.

💡 **Why it matters:** Shows up in **robot movement, rotating buffers, and ring systems**.

🔄 Variants:

- Circular array shifts
- Round-robin cycles
- Clock logic

🔥 Examples:

- [Leetcode 189 - Rotate Array](#)
 - [Leetcode 1041 - Robot Bounded in Circle](#)
 - [Leetcode 795 - Number of Subarrays with Bounded Maximum](#)
-

107. 🧠 Simulation – Turn-by-turn System

📌 Simulate **step-by-step system changes** like a game or queue.

🎯 **Core Idea:** Use arrays or queues to mimic what happens over time or turns.

💡 **Why it matters:** Important in **scheduling, gaming, elimination systems**.

🔄 Variants:

- Josephus elimination
- Political turn-taking
- Turn-based queue simulation

🔥 Examples:

- [Leetcode 649 - Dota2 Senate](#)
- [Leetcode 1823 - Find the Winner of the Circular Game](#)
- [Leetcode 1352 - Product of the Last K Numbers](#)

SECTION 11 : RARE BUT USEFUL WILDCARD PATTERNS

107. Median of Two Sorted Arrays (Log(M+N) Merge Trick)

 Find the median of two sorted arrays in $O(\log(\min(m, n)))$ time.

 **Core Idea:** Binary search the shorter array to partition both arrays correctly.

 **Why it's important:**

- It's one of the most classically **deceptive hard problems** — simple concept, but the correct solution is tricky.
- Frequently asked at **Google and Facebook** to test binary search depth and boundary handling.

 FAANG Example:

- [Leetcode 4 - Median of Two Sorted Arrays](#)

108. Expression Evaluation (Infix to Postfix + Stack Evaluation)

 Evaluate math expressions with $+ - * /$, parentheses, and proper precedence.

 **Core Idea:** Use the **shunting yard algorithm** to convert infix to postfix, then evaluate using a stack.

 **Why it's important:**

- Tests **stack mastery + parsing logic**.
- Common in **parser design, calculator simulations, and compiler interviews** (Google, Dropbox).

 FAANG Examples:

- [Leetcode 224 - Basic Calculator](#)
- [Leetcode 772 - Basic Calculator III](#)

109. Number to Words (Recursive Chunking + Mapping)

 Convert a number like `12345` to `"Twelve Thousand Three Hundred Forty Five"`.

🎯 **Core Idea:** Divide the number into 3-digit chunks, and recursively handle special English number rules.

💡 **Why it's important:**

- Tests **attention to edge cases**, formatting, and recursion control.
- Seen in **Apple, Microsoft**, and formatting-heavy roles.

🔥 FAANG Example:

- [Leetcode 273 - Integer to English Words](#)
-

110. 🧠 Max Frequency Character in a Sliding Window

📌 Track the character with **highest frequency** in a moving window.

🎯 **Core Idea:** HashMap to count chars, and a variable to track max frequency.

💡 **Why it's important:**

- Shows up in **character replacement, compression, and decoding** problems.
- Interviewers want to see if you can maintain state efficiently across a window.

🔥 FAANG Example:

- [Leetcode 424 - Longest Repeating Character Replacement](#)
-

111. 🧠 Random Pick With Weight (Prefix Sum + Binary Search)

📌 Pick a random index from an array where **each index has weight** (likelihood).

🎯 **Core Idea:** Build a prefix sum array and use binary search on a random number from `[1, total weight]`.

💡 **Why it's important:**

- Appears in **machine learning system design, ads, recommendation engines**.
- Asks: can you map **continuous probability** into **discrete index selection**?

🔥 FAANG Example:

- [Leetcode 528 - Random Pick with Weight](#)
-

112. 🧠 Interval Tree / Dynamic Interval Tracking

📌 Maintain a stream of intervals and track **merging, insertion, and disjoint states**.

🎯 **Core Idea:** Use a **balanced tree (TreeMap or BST)** to store and merge intervals in real time.

💡 **Why it's important:**

- Used in **calendar systems, database query windows, memory allocators**.
- Shows **mastery of TreeMap-like structures and edge cases**.

🔥 FAANG Example:

- [Leetcode 352 - Data Stream as Disjoint Intervals](#)
-

113. 🧠 Binary Indexed Tree (Fenwick Tree)

📌 Efficiently perform **prefix sum and updates** in $O(\log n)$.

🎯 **Core Idea:** Use clever bit manipulation to build a tree-like array.

💡 **Why it's important:**

- Appears in **competitive programming and range query problems**.
- Compact and elegant alternative to Segment Tree.

🔥 FAANG Example:

- [Leetcode 307 - Range Sum Query - Mutable](#)
 - [Leetcode 315 - Count of Smaller Numbers After Self](#)
-

114. 🧠 Sparse Table (Immutable Range Queries)

📌 Preprocess array to answer **min/max/range queries** in $O(1)$ after $O(n \log n)$ setup.

🎯 **Core Idea:** Store answers for overlapping ranges of length 2^k .

💡 **Why it's important:**

- Used in **LCA (Lowest Common Ancestor)** problems and range min/max without updates.
- Great for static data where speed is critical.

🔥 Competitive Programming Use:

- Range Minimum Query
 - LCA preprocessing in trees
-

115. 🧠 Meet in the Middle

📌 Split the input into two halves and solve each **independently**, then combine.

🎯 **Core Idea:** Balance brute-force on both halves to reduce time from $O(2^n)$ to $O(2^{(n/2)})$.

💡 **Why it's important:**

- Works where input size is small but too big for direct recursion.
- A **hidden gem** technique used in **Google coding interviews**.

🔥 FAANG Examples:

- [Leetcode 494 - Target Sum](#)
 - [Leetcode 1125 - Smallest Sufficient Team](#)
-

116. 🧠 Monotonic Queue (Sliding Window Optimization)

📌 Optimize sliding window problems by maintaining a queue that is **monotonically increasing/decreasing**.

🎯 **Core Idea:** Remove elements from back that will never be used again.

💡 **Why it's important:**

- Gives **$O(n)$** time solutions to max/min sliding window problems
- Showcases **thinking in terms of invariants** — a key skill in system design too.

🔥 FAANG Example:

- [Leetcode 239 - Sliding Window Maximum](#)
 - [Leetcode 862 - Shortest Subarray with Sum at Least K](#)
-

117. 🧠 Trie + Bitmask Hybrid

📌 Combine **prefix tree** with bitmask state tracking for **high-performance prefix + state problems**.

🎯 **Core Idea:** Build a trie from words and use bitmasks for constraints (like used letters).

💡 **Why it's important:**

- Unlocks problems like **Word Search**, **XOR maximization**, and **constraint pruning**.
- Seen in advanced **Google + Dropbox** questions.

🔥 FAANG Examples:

- [Leetcode 421 - Max XOR of Two Numbers](#)
 - [Leetcode 212 - Word Search II](#)
-

118. 🧠 Offline Union-Find (Dynamic Connectivity)

📌 Perform **connectivity operations backwards** using union-find to speed up queries.

🎯 **Core Idea:** Process the events in reverse — deletions become additions.

💡 **Why it's important:**

- Used in **rollback systems**, version control, history tracking.
- Loved in **hardcore design rounds** and Google deep-dives.

🔥 FAANG Example:

- [Leetcode 1202 - Smallest String With Swaps](#)
- [Codeforces/CP - Offline Dynamic Connectivity](#)

FINAL BONUS SECTION 12: DESIGN / OOP-BASED CODING PATTERNS

Each pattern below reflects a **real system behavior** that needs to be **modeled using classes, data structures, and logic**. These aren't just about passing test cases — they show how well you **simulate real-world systems** using code.

119. 🧠 LRU Cache (Least Recently Used Cache)

📌 Design a cache that evicts **least recently used** items when full

🎯 Core Idea: Combine **HashMap + Doubly Linked List** for O(1) get and put

💡 Why: Simulates memory/cache logic in OS/browser

🔄 Variants:

- LFU Cache (least frequently used)
- Sliding TTL cache
- LRU + expiry time

🔥 FAANG Examples:

- [Leetcode 146 - LRU Cache](#)
 - [Leetcode 460 - LFU Cache](#)
 - [GFG - LRU Cache Implementation](#)
-

120. 🧠 Rate Limiter / Token Bucket Design

📌 Limit number of API calls **per user/service/time window**

🎯 Core: Use **Queue or Sliding Window + Timestamp logic**

💡 Why: Common in backend systems, APIs, and payment systems

🔄 Variants:

- Fixed window
- Sliding window
- Token bucket / Leaky bucket

🔥 FAANG Examples:

- [GFG - Rate Limiter using Sliding Window](#)
 - [System Design Round - Build Rate Limiter](#)
 - [Design Uber Surge Pricing Rate Controller](#)
-

121. 🧠 In-Memory File System

📌 Build a system with **mkdir, ls, read, write, path traversal**

🎯 Core Idea: Use **Trie-like structures + directory trees**

💡 Why: Tests tree design + class abstraction

🔄 Variants:

- Folder compression
- Snapshot versioning
- Permissions modeling

🔥 FAANG Examples:

- [Leetcode 588 - Design In-Memory File System](#)
 - [Design Dropbox file explorer](#)
 - [Design Git file tree](#)
-

122. 🧠 Logger Rate Limiter

📌 Allow logging only if message hasn't been logged in X seconds

🎯 Core: Use a **HashMap with timestamp tracking**

💡 Why: Popular for log de-duplication, alert throttling

🔄 Variants:

- Per-service logs
- Multiple rate levels (INFO, WARN, ERROR)

🔥 FAANG Examples:

- [Leetcode 359 - Logger Rate Limiter](#)

- [System Design Round - Alert Manager](#)
-

123. 🧠 Parking Lot Design

📌 Model parking floors, slots, vehicles, and ticketing

🎯 Core: Use enums, interfaces, slot assignment strategies

💡 Why: Great to test class design + interface separation

🔄 Variants:

- Different slot types
- Dynamic pricing
- Car vs Bike vs Truck

🔥 FAANG Examples:

- [System Design Interview - Design a Parking Lot \(Uber, Amazon\)](#)
 - [LLD Round - Oyo / GoJek](#)
-

124. 🧠 Snake Game / Tetris / Maze Simulation

📌 Model movement of objects in grid-based systems

🎯 Core: Use Deque/Set/Queue for state tracking

💡 Why: Tests 2D spatial logic, self-collision, simulation loops

🔄 Variants:

- Snake eating food
- Tetris block stacking
- Maze escape, Pacman style

🔥 FAANG Examples:

- [Leetcode 353 - Design Snake Game](#)
 - [Leetcode 1275 - Find Winner on a Tic Tac Toe Game](#)
 - [Leetcode 490 - The Maze](#)
-

125. 🧠 Design Twitter / Instagram Feed

📌 Implement posting, following, and news feed generation

🎯 Core: Use **Heap, Graph, Queue, Map of Queues**

💡 Why: Combines social graphs with stream merge

🔄 Variants:

- Top 10 latest posts
- Influencer detection
- Like/share/comment system

🔥 FAANG Examples:

- [Leetcode 355 - Design Twitter](#)
 - [Leetcode 432 - All O\(1\) Data Structure](#)
 - [Design Instagram with Story/Feed APIs](#)
-

126. 🧠 Design Elevator System

📌 Simulate multiple elevators, request queues, direction logic

🎯 Core: Use OOP to model requests, movement, state transitions

💡 Why: Tests concurrency modeling + queuing

🔄 Variants:

- Peak hour strategy
- Load-balancing elevators
- Priority passengers

🔥 FAANG Examples:

- [System Design - Elevator Control System](#)
 - [Uber / Dropbox Design Round](#)
-

127. 🧠 Design Scheduler / Job Queue

📌 Execute jobs based on delay, interval, or time

🎯 Core: Use min-heap, delay queue, or priority queue

💡 Why: Great for modeling OS schedulers

🔄 Variants:

- Job retries
- Delayed vs interval jobs
- Job dependencies

🔥 FAANG Examples:

- [Leetcode 621 - Task Scheduler](#)
- [Leetcode 358 - Rearrange String K Distance Apart](#)
- [GFG - Design Job Scheduler](#)